

Free Open-Source Engine Turns AI Agent Workflows Into Verified Contracts

circuit, from Punt Labs, replaces prompt-based process discipline with small, model-checked state machines. The agent may request progress; the machine decides whether progress is valid.

Every workflow gate runs the team's own commands, so "tests pass" means the suite passed.

Press Release

SAN FRANCISCO, March 1, 2027: Punt Labs today announced circuit, a free, open-source workflow engine for engineering teams that run AI coding agents unattended on repositories already guarded by CI. Teams write a workflow such as test-driven development as a small formal state machine and prove it sound before first use. At runtime the machine, never the agent, decides when the workflow advances. Instruction files, skills, and hooks advise an agent and hope it complies. circuit enforces: each step is gated on evidence from commands such as the test suite, the linter, and CI status (see [FAQ 2](#)). Teams get agent loops they can leave unattended, and a trace that proves the process was followed.

Problem

Engineering teams that adopt AI coding agents encode their process in prose. Instruction files, skills, hooks, and ever-longer prompts say "write the failing test first," "never merge until review is clean," "run the full gate before committing." Agents follow these instructions most of the time, and drift the rest of the time. They skip the failing-test step, declare success while the build is red, and treat their own summary as proof the work is done. Each drift costs a review cycle to catch and a rework cycle to fix. It also caps autonomy, because a loop that cannot be trusted must be watched. Current fixes sit at two extremes. More prose makes the context longer and the compliance no more certain; every soft mechanism relies on the agent choosing to obey. Governance platforms add identity, delegation contracts, and audit trails. The team adopts an entire organizational model to get one guarantee: the workflow is followed.

Solution

With circuit, a team writes the workflow once, as a state machine of a few dozen lines: named states, allowed transitions, and the facts each transition requires. During development, a model checker verifies the machine, so an unreachable state or a rule conflict is caught before an agent ever runs the loop. At runtime, a single small binary interprets the machine with no server, no cloud, and no verification tooling required. Each gating fact is bound to a command. A transition that requires passing tests runs the test suite; one that requires clean review reads the review state itself. When an agent asks to advance, circuit advances or refuses, naming exactly which condition failed. The refusal lands in the agent's context as its next instruction. In drive mode, circuit runs the loop itself. It prompts the agent with the current state and valid next steps, and validates every requested transition (see [FAQ 4](#)). It also writes a step-by-step trace, so the team can read afterward what the agent did in each state and why each gate opened.

Customer Quote

“We had a beautifully written TDD instruction file, and our agents still wrote the implementation first whenever the test was awkward. With circuit the implement step simply refuses until a failing test exists, and the agent routes around its own laziness instead of ours. Last month I started a quality-ratchet loop before a long weekend and came back to nine merged refactors and a trace file for each one. I read traces over coffee now instead of babysitting terminals.”

— **Maya Okafor**, Staff Engineer at Fieldline Robotics

Getting Started

Getting started takes three steps and no infrastructure. First, install the single binary and run `circuit list` to see the shipped machines, including test-driven development and a pull-request watch loop. Second, run `circuit scaffold` in a repository. The scaffold generates stub bindings that connect the machine’s facts to that project’s own commands. A stub defaults to false, so an unfinished integration blocks safely instead of passing silently. Third, start the loop: let an agent harness call circuit as a tool, or run `circuit drive` and let circuit steer the agent directly. A team can watch a machine refuse an invalid transition within ten minutes of installing, with no server and no account.

Spokesperson Quote

“Everyone is trying to fix agent process drift with better prose, and prose is advice. The agent grades its own homework. Our design bet was to take the workflow out of the prompt entirely and put it in a tiny formal machine. Prove it correct at development time, keep the runtime small and boring, and gate every step on a command the agent cannot fake. The pattern is decades old; safety-critical rail systems have run on proven state machines since the nineties. What surprised us in our own use was the behavior change. When the quality gate refused to open, the agent stopped arguing it was done and went back to refactoring until the gate passed. Enforcement turned out to be better feedback than instruction.”

— **Jim Freeman**, Founder at Punt Labs

Call to Action

circuit is available today at <https://github.com/punt-labs/circuit>, free under the MIT license. It ships as one Go binary with five ready-to-run workflow machines. It works with any agent harness that can call a command-line tool. Start with the ten-minute walkthrough in the repository README.

Frequently Asked Questions

External FAQs

Q1: What is circuit and who is it for?

circuit is a free workflow engine that makes an AI coding agent follow a defined process. It is for engineering teams that run agents such as Claude Code, Cursor, or pi unattended, on repositories where CI gates already decide what merges. The team writes the workflow as a small state machine, and circuit refuses any step whose required evidence is missing. The core job: run an agent loop unattended without trusting the agent's own report of success.

Q2: Why are skills, instruction files, and hooks not enough?

Prose instructions depend on the agent choosing to comply, and measurement shows compliance drifts. Across 20,574 coding-agent sessions in the wild, developer constraint violation was the largest misalignment symptom at 38.33%, and instruction-following failure was the largest root cause at 36.49% [1]. Better prompting narrows the gap without closing it: the failure pattern is architectural rather than a wording defect [2]. Hooks intercept single tool calls, so they cannot see whether a multi-step workflow ran in order. circuit sits outside the model. A transition either satisfies its guard or it does not, and no phrasing changes the answer.

Q3: How is circuit different from LangGraph?

LangGraph compiles an agent workflow into a state graph and validates the graph's structure before execution [3]. Its transitions are still decided by model outputs, so the system being constrained also judges the constraint. circuit decides transitions from externally executed commands: the test suite, the linter, CI status. circuit is also engine-shaped rather than framework-shaped. You keep your existing harness and add one binary; you do not rewrite your agent as a graph.

Q4: Why B-Method state machines instead of a YAML workflow file?

A YAML file can list states and transitions; it cannot tell you the workflow is sound. B machines are model-checked during development, so deadlocks, unreachable states, and rule conflicts surface before an agent ever runs the loop. The method has industrial history: the automatic train control for Paris Métro Line 14 was developed in B, with no bug found in over 25 years of revenue service [4]. Amazon reported comparable payoff from formal specification of AWS designs [5]. Runtime stays cheap: a small Go interpreter reads the machine, and the verification tooling is a development dependency only. One platform limit applies: the verifier ships no ARM build, so on Apple Silicon or ARM Linux the design-time check runs in CI or an x86_64 container rather than on the laptop. The runtime binary runs anywhere Go does.

Q5: Do I need to know B or formal methods to use circuit?

Not to use the shipped machines. `circuit list` shows them; `circuit scaffold` wires one to your repository's commands; you never open the machine file. Authoring a new machine means writing a few dozen lines in a strict, small profile of B, about the effort of writing a Makefile. That comparison is our own estimate, untested outside Punt Labs; the interviews in [FAQ 16](#) include an unassisted authoring exercise to test it. Unsupported constructs fail with a diagnostic that points at the offending line.

Q6: How do I get started?

Install the single binary. Run `circuit list` to pick a machine such as `tdd-flow` or `pr-watch`. Run `circuit scaffold` to generate check bindings for your repository, then start the loop from your harness or with `circuit drive`. The first refused transition, with its named failing condition, appears within about ten minutes.

Q7: How much does it cost?

Nothing. `circuit` is MIT-licensed open source with no paid tier. You pay only for the agent and model you already run.

Q8: What happens to my code and data?

`circuit` runs entirely on your machine. There is no server, no account, and no telemetry. Session state lives in JSON files inside your repository's temporary directory, and traces stay local unless you choose to share them.

Q9: What happens when a gate is wrong, or an emergency requires bypassing the process?

Stop the session and act by hand; `circuit` constrains the agent's loop, never your shell. The durable fix is an edit to the machine or its bindings, which are text files that travel through code review like any other change. The workflow itself becomes versioned and reviewable.

Q10: What does a false refusal cost, and how do we keep gates honest?

A wrong gate blocks valid work, and a process tool that blocks valid work gets removed. That abandonment path is the most plausible long-term failure, so it gets its own instrumentation. Every refusal names the failed condition and the command behind it, so diagnosis is one trace line, and the repair is a small reviewed edit to the machine or a binding. We track override rate next to interception rate ([FAQ 23](#)): sessions stopped by hand, and gates loosened to get past a refusal. A rising override rate means the machine is refusing valid work, and we treat that as a defect with the same urgency as a red build.

Internal FAQs

Value & Market

Q11: What is the total addressable market?

No survey measures demand for an agent-workflow enforcement layer, so every figure here is a labeled assumption built on adjacent data. By January 2026, 90% of surveyed developers used an AI tool at work [6]; by mid-2026, 68% used an AI agent daily [7]; meanwhile only 29% of developers said they trust AI output to be accurate [8]. Our assumed serviceable segment: teams that already run agents against CI on repositories they must keep healthy, a strict subset of daily-agent users with existing gate discipline. The chain, with every step a labeled assumption: roughly 30 million professional developers worldwide, itself an order-of-magnitude assumption in line with common industry population estimates. Assume the surveyed agent-daily share generalizes to one third of them: about 10 million. Assume one in ten works on a team with CI gate discipline: about 1 million. Assume a 2 to 3% adoption share: 20,000 to 30,000 developers. That arithmetic is a hypothesis, and the step in [FAQ 16](#) exists to replace it with observed numbers.

Q12: What evidence do we have that customers want this?

Primary customer evidence does not exist yet. The only customer to date is Punt Labs itself, which we flag as the classic high-risk signal. The problem, by contrast, is well measured: instruction-following failure and inaccurate self-reporting are dominant failure categories across 20,574 in-the-wild sessions [1], and the failure pattern is architectural rather than fixable by prompting [2]. Our dogfood results: circuit-driven TDD slices delivered merged production changes; a failing quality gate forced repeated refactor loops until it passed; a pr-watch machine gated a live pull request through fix, review, and merge. In the wild, visible resolution of agent misalignment usually requires explicit developer pushback [1]. circuit automates the subset of that pushback expressible as a command's exit status. The mechanism claim itself rests on inference plus our own dogfood observation: no controlled study yet compares externally gated agent work against self-assessed agent work, and running that comparison on our published traces is part of the validation plan in [FAQ 16](#).

Q13: Who are the competitors, and why will we win?

Sponsio is the closest product: it compiles policy into deterministic finite-state contracts checked at every tool call, with no model in the enforcement path [9]. Its formal layer is LTL automata for general tool-call governance; its public materials describe no design-time model checking, and it does not target coding workflows such as TDD or PR cycles. LangGraph owns mindshare for graph-shaped orchestration, and its transitions are judged by model outputs [3]. One research prototype model-checks an agent runtime in TLA+ [10], an early signal of the direction. Honest assessment: any of these players could copy the pattern within a quarter. Our defensible ground is narrow and specific: a formal method proven in rail service [11], model checking required at design time, checks bound to repository-authored commands, and neutrality across harnesses (see [Feature 16](#)). We win by being the smallest correct piece, shipping working machines for the loops teams run every day. The 90-day question, answered plainly: if Sponsio reads this press release tomorrow and ships TDD and PR-cycle contract templates, nothing structural remains, only a head start on coding workflows and the B toolchain. The bet is focus over structural advantage, and it is the same bet the Value rating marks HIGH.

Q14: How does circuit relate to ethos and beadle?

They are complementary layers, and ethos is the heavier one by design. ethos answers who may do what: identity, delegation contracts, write-sets, audit. circuit answers when work may advance: a process gate with no organizational model attached. A team can adopt circuit alone in an afternoon. Internally the integration runs the other way. ethos mission pipelines are informal state machines today and can execute on circuit machines. beadle's planned autonomous mail daemon needs exactly circuit's rule, advance only when trust checks pass, before it acts on a message. One engine, three surfaces.

Q15: What is the customer acquisition strategy?

Distribution follows the harness ecosystems: a pi extension exists today, a Claude Code adapter follows, and the shipped machines double as demos. Traces make sharable evidence; a blocked-transition log is a better pitch than a benchmark. Punt Labs' own tools are the first embedders (FAQ 14), so ethos and beadle installs carry circuit's pattern with them. Channels, costs, and conversion rates are assumptions and remain unvalidated at this stage.

Q16: What is your next step to validate the vision?

Interview ten engineering teams that run coding agents unattended, and count how many have already built ad-hoc gating scripts around their agents. An existing workaround a team maintains is the strongest demand signal available. In parallel, run pr-watch and tdd-flow on every Punt Labs repository for one month and publish the traces, including every refused transition. If the interviews find no workaround-builders and our own traces show no interceptions, the value hypothesis fails and we stop. The dogfood month also logs every override, because a team that adopts the tool and then disables it is the third failure mode (FAQ 10).

Technical**Q17: What are the major technical risks?**

Check independence is the sharpest risk. A gate is only as trustworthy as the independence of what it runs. In one study of LLM-based test generators, up to 68.1% of one tool's final suites on the Refactory dataset passed on incorrect implementations, with broader averages near 26 to 35% [12]. Mitigation: bindings resolve to repository-authored commands under code review, and the registry accepts no agent-authored checks mid-session. Second, profile expressiveness: a strict B subset may prove too narrow for messy workflows. Mitigation: guards combine and negate externally observed facts, and anything beyond the profile lands in a reviewed check command rather than in the machine language. Third, harness churn: adapters track moving surfaces such as pi's RPC mode. Mitigation: adapters stay thin, the CLI is the contract, and the engine itself has no harness dependency.

Q18: What dependencies exist, and what is the fallback?

ProB is a development and release dependency only, pinned at 1.15.1 through Nix; runtime needs nothing beyond the Go binary. ProB publishes no Linux aarch64 build, so machine verification runs on x86_64 in CI. Drive mode currently depends on pi's RPC mode. The fallback relationship works everywhere: any harness that can run a command-line tool can host circuit.

Q19: What is the delivery order, and what gets cut first?

Delivery order, without dates. First, harden the engine and the five shipped machines, and make per-run prompt overrides first-class input. Second, ship the Claude Code adapter, where the widest audience already works. Third, embed circuit in ethos mission pipelines and beadle's daemon (FAQ 14). Fourth, run an external beta with the interviewed teams. If scope compresses, the external beta and the beadle embedding cut first. The engine and the model-check gate never cut.

Q20: What breaks first if this succeeds?

The engine breaks last; interpreting a machine of a few dozen lines is negligible work. Authoring breaks first: at many machines per organization, writing and reviewing B by hand needs templates and better diagnostics. Check registries sprawl second: per-repository command bindings will drift and want linting. Session visibility is third: JSON files per repository serve one team well and stop short of any fleet-level view. Each is tooling work, and none threatens the core design.

Business**Q21: What is the revenue model?**

None, deliberately. circuit is free, MIT-licensed infrastructure, the same posture as ethos and beadle. The strategic value: it is the enforcement primitive the rest of the Punt Labs toolchain stands on, and the cheapest credible proof that our agent tooling holds itself to verifiable process. Costs are maintenance attention only; there is no infrastructure to operate. If a commercial layer ever exists, it sits above the engine, in fleet visibility or hosted machine authoring, and the engine stays free.

Q22: What does circuit cost us to run?

Maintenance attention, made explicit. The owner is Punt Labs' founder, with agent tooling carrying routine upkeep. The assumed steady-state load is a few hours a week outside integration pushes, an order-of-magnitude guess we will replace with observed effort. Support arrives as GitHub issues with no service commitment, and the no-telemetry choice (Feature 17) means we never see the failures nobody reports. The displaced alternative is time on ethos and beadle, which the embeddings in FAQ 19 partially return.

Q23: What are the key metrics, and what makes us reconsider?

Four metrics, each with a decision threshold. Interception rate: refused transitions per driven session, read from traces. If a month of dogfood shows near-zero interceptions, the layer guards against a problem we do not have, and we reassess. Interceptions alone cannot prove health, since a machine that refuses everything scores perfectly, so the metric pairs with override rate. Override rate: sessions stopped by hand or gates loosened to get past a refusal (FAQ 10). If overrides exceed one in five refusals, a starting threshold we will recalibrate against observed runs, the gates are wrong, and machine fixes come before new machines. Unattended completion: driven loops reaching a terminal state without human intervention, targeting a majority of runs, since babysitting is the cost we set out to remove. External embedding: repositories outside Punt Labs with a committed machine file. If that count is still zero two quarters after the Claude Code adapter ships, the open-source bet needs rework.

Q24: Why now?

Agent adoption and agent trust are moving in opposite directions. Daily agent use reached 68% of surveyed developers in the May to July 2026 JetBrains wave [7]. Trust in AI output stood at 29% in the May to June 2025 Stack Overflow survey, the lowest it has recorded [8]. A controlled study found experienced developers roughly 19% slower with AI assistance while believing themselves faster [13], so self-perception is unreliable on both sides of the keyboard. Enforcement architectures are emerging in research and product [10, 9], and none yet owns coding-workflow discipline. The window is the gap between agents everywhere and guarantees nowhere.

Q25: What are we not building?

No scheduler: the harness or the operator owns cadence. No GitHub client, no MCP server, and no persistence layer inside the engine; observations arrive through bound check commands. No general B interpreter: the strict profile is the product. No model in the enforcement path, ever. No hosted service and no telemetry. Each exclusion keeps the runtime small enough to trust and to review.

Risk Assessment

Risk	Rating	Assessment
Value	HIGH	The problem is externally measured, but demand for an enforcement layer is unmeasured, and the only customer so far is Punt Labs itself. The competitive moat is focus, with nothing structural behind it (FAQ 13). The interviews in FAQ 16 test exactly this.
Usability	MEDIUM	Using a shipped machine is a three-step start. Authoring a new machine requires B literacy, and no analogous product gives customers a mental model.
Feasibility	MEDIUM	The engine, five machines, drive mode, traces, and CI gates run today, and that scope is low risk. The announced product also needs the un-built Claude Code adapter, drive beyond pi, and an answer to check independence (FAQ 17).
Viability	LOW	Free open source with no operating cost. The remaining cost is maintenance attention, owned and estimated in FAQ 22, and revisited at each metric threshold in FAQ 23.

Feature Appendix

Must Do

- F1. Circuit-B engine:** a strict, multi-pass parser and evaluator with diagnostics anchored to the author's source; the machine is the sole authority on valid progress
- F2. Model-check gate:** every shipped machine passes ProB initialization and model checking before release
- F3. Check bindings:** machine facts bound to repository-authored commands; scaffolded stubs default to false so an incomplete integration blocks safely
- F4. Multi-session runtime:** suspend and resume across short CLI invocations, independent sessions per machine, auto-stop on terminal states
- F5. Drive mode with traces:** circuit owns the loop, prompts the agent per state, validates every requested transition, and writes a step-by-step trace
- F6. Shipped machines:** tdd-flow, pr-watch, review-flow, retry-flow, and build-job as ready-to-run workflow contracts

Should Do

- F7. Claude Code adapter:** native commands and per-turn context injection for the largest harness audience
- F8. Per-run prompt overrides:** named prompt presets as run input rather than worktree file edits, unblocking ratchet-style runs
- F9. ethos embedding:** mission pipelines execute on circuit machines instead of informal stage logic
- F10. beadle embedding:** the autonomous mail daemon advances only through trust-gated transitions
- F11. Richer traces:** capture check command output and machine-readable refusal reasons alongside transitions
- F12. Worktree preparation tooling:** one command yields a ready worktree, so a gate failure reflects agent work rather than environment setup

Won't Do

- F13. Scheduler:** cadence belongs to the harness or the operator; a built-in scheduler grows circuit into the platform it refuses to be
- F14. Built-in GitHub, MCP, or persistence integrations:** observations arrive through bound check commands; direct integrations would couple a tiny engine to moving external APIs
- F15. Full B-Method support:** the strict profile keeps the evaluator small, reviewable, and testable; general B belongs to ProB
- F16. Any model in the enforcement path:** a language model judging transitions reintroduces the self-report problem circuit exists to remove
- F17. Hosted service or telemetry:** trust in an enforcement tool starts with it running entirely on the team's own machines

References

- [1] Ningzhi Tang et al. *How Coding Agents Fail Their Users: A Large-Scale Analysis of Developer-Agent Misalignment in 20,574 Real-World Sessions*. Observational study of 20,574 real coding-agent sessions across 1,639 repositories (Cursor, GitHub Copilot, Claude Code, Codex, OpenCode, Gemini CLI); quantifies instruction-following failure, developer constraint violation, and inaccurate self-reporting as recurring misalignment symptoms. 2026. URL: <https://arxiv.org/abs/2605.29442>.
- [2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. *Why Do Multi-Agent LLM Systems Fail?* Taxonomy of 14 multi-agent LLM failure modes (MAST) from 1,642 execution traces across 7 frameworks; argues failures are architectural, not solvable by base-model improvement alone. 2025. URL: <https://arxiv.org/abs/2503.13657>.
- [3] LangChain. *LangGraph Overview*. A compiled LangGraph StateGraph is a finite state machine whose transitions are driven by LLM outputs and tool results; graph compilation validates connections and detects cycles before execution, but transition validity is decided by the same LLM outputs being constrained. 2026. URL: <https://docs.langchain.com/oss/python/langgraph/overview> (visited on 08/30/2026).
- [4] CLEARSY. *Extension of Line 14 of the Paris Metro: Over 25 Years of Reliability Thanks to the B Formal Method*. Météor (Paris Métro Line 14) automatic train control software: over 110,000 lines of B model generating 86,000 lines of Ada; no bug found after proof completion through validation, integration, on-site testing, or 25+ years of revenue service. 2024. URL: <https://www.clearsy.com/en/the-tools/extension-of-line-14-of-the-paris-metro-over-25-years-of-reliability-thanks-to-the-b-formal-method/> (visited on 08/30/2026).
- [5] Chris Newcombe et al. “How Amazon Web Services Uses Formal Methods”. In: *Communications of the ACM* 58.4 (2015). Primary account by AWS engineers: seven teams used TLA+ by 2015; TLA+ model checking found subtle DynamoDB design bugs missed by design review, code review, and testing. Executive management began proactively encouraging TLA+ specs afterward. URL: <https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf>.
- [6] JetBrains. *Which AI Coding Tools Do Developers Actually Use at Work?* AI Pulse survey, 10,000+ professional developers, January 2026 wave: 90% use an AI tool at work; GitHub Copilot 29%, Cursor 18%, Claude Code 18% workplace adoption. 2026. URL: <https://blog.jetbrains.com/research/2026/04/which-ai-coding-tools-do-developers-actually-use-at-work/> (visited on 08/30/2026).
- [7] JetBrains. *AI Coding Agents: Adoption Trends*. May-July 2026 survey wave: Copilot workplace adoption falls to 21%, Cursor to 12%; 68% of developers use an AI agent daily; OpenCode reaches 7% adoption. 2026. URL: <https://blog.jetbrains.com/research/2026/08/ai-coding-agent-adoption-2026/> (visited on 08/30/2026).
- [8] Stack Overflow. *2025 Stack Overflow Developer Survey*. 49,009 responses, 166 countries, fielded May-June 2025. 84% use or plan to use AI tools (up from 76% in 2024); only 29% trust AI output to be accurate (down from 40% in 2024); 66% report AI solutions are frequently close but wrong. 2025. URL: <https://survey.stackoverflow.co/2025/ai> (visited on 08/30/2026).

- [9] Sponsio. *Sponsio: Runtime Contract Enforcement for AI Agents*. Commercial product compiling natural-language policy into deterministic finite-state-machine (LTL) contracts checked at every agent tool call, with no LLM in the enforcement path. Closest existing product to a machine-decides-not-agent-decides architecture; formal layer is LTL/finite automata, not a full B-Method abstract-machine specification. 2026. URL: <https://sponsio.dev/> (visited on 08/30/2026).
- [10] Riddhi Mohan Sharma. *Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems*. Sole-author preprint, not peer reviewed. TLA+ specification of a zero-trust agent runtime with a PERMIT/DENY/ESCALATE safety invariant; the TLC model check is bounded (1,738 states, 324 distinct, depth 8). 2026. URL: <https://arxiv.org/abs/2605.17909>.
- [11] Thierry Lecomte et al. “Applying a Formal Method in Industry: A 25-Year Trajectory”. In: *Formal Methods: Foundations and Applications (SBMF 2017)*. Vol. 10623. Lecture Notes in Computer Science. Retrospective on the B-Method’s industrial trajectory since Météor Line 14, including B’s adoption by Alstom’s CBTC product line (roughly 100 metro lines worldwide). 2017, pp. 70–87. URL: <https://arxiv.org/abs/2005.07190>.
- [12] Noble Saji Mathews and Meiyappan Nagappan. *Design Choices Made by LLM-Based Test Generators Prevent Them from Finding Bugs*. Evaluation of GitHub Copilot, Codium CoverAgent, and CoverUp on the Refactory dataset (2,442 correct and 1,783 buggy human-written Python programs); up to 68.1% of one tool’s final generated test suites passed on incorrect implementations while failing on correct ones, with broader averages near 26–35% depending on whether the model saw buggy code. 2024. URL: <https://arxiv.org/abs/2412.14137>.
- [13] METR. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. Randomized controlled study: AI coding assistance made experienced developers roughly 19% slower on real tasks despite perceiving themselves as roughly 20% faster. 2025. URL: <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>.